

线性注意力机制：原理与实践

WPC

2026 年 5 月 27 日

目录

第一章 注意力机制基础	2
1.1 引言	2
1.2 标准注意力公式	2
1.3 多头注意力	2
1.4 计算复杂度分析	3
1.5 长序列挑战	3
第二章 线性注意力的动机	4
2.1 二次复杂度的根本问题	4
2.2 为什么不能简单优化矩阵乘法?	4
2.3 结合律的启发	4
2.4 Softmax 的挑战	5
2.5 线性注意力的基本思路	5
第三章 线性注意力的核心原理	6
3.1 核函数视角的注意力	6
3.2 线性注意力公式	6
3.3 计算顺序的变换	7
3.4 特征映射的设计	7
3.4.1 elu+1 映射	7
3.4.2 随机傅里叶特征	8
3.5 因果注意力与掩码	8
第四章 Linear Transformer	9
4.1 概述	9
4.2 从 softmax 到可分解核	9
4.3 归一化	9
4.4 因果掩码的递推形式	10
4.5 多头线性注意力	10
4.6 复杂度对比	10

4.7	伪代码	11
第五章	Performer 与随机特征注意力	12
5.1	概述	12
5.2	FAVOR 机制	12
5.2.1	softmax 核的随机特征近似	12
5.2.2	正交随机特征	13
5.3	FAVOR+ 的具体实现	13
5.4	理论保证	13
5.5	与其他线性注意力的比较	13
5.6	训练技巧	14
第六章	其他线性注意力变体	15
6.1	概述	15
6.2	MGA (Memory-based Global Attention)	15
6.3	GLA (Gated Linear Attention)	15
6.4	RWKV	15
6.5	Retentive Network (RetNet)	16
6.6	HGRN (Hierarchically Gated RNN)	16
6.7	基于卷积的线性注意力	16
6.8	变体比较	17
第七章	线性注意力的挑战与改进	18
7.1	表达能力的差距	18
7.1.1	问题分析	18
7.1.2	实验观察	18
7.2	数值稳定性问题	18
7.2.1	分母接近零	19
7.2.2	状态矩阵的累积	19
7.3	改进方法	19
7.3.1	混合注意力	19
7.3.2	改进的特征映射	19
7.3.3	训练策略改进	20
7.4	理论分析	20
7.4.1	近似误差界	20
7.4.2	表达能力分析	20
7.5	实践建议	20

第八章 总结与展望	22
8.1 线性注意力的核心贡献	22
8.2 主要方法回顾	22
8.3 与 Mamba/SSM 的关系	22
8.4 未来方向	23
8.5 结语	23

第一章 注意力机制基础

1.1 引言

注意力机制 (Attention Mechanism) 是现代深度学习的核心技术之一，尤其在自然语言处理领域取得了巨大成功。本节回顾标准注意力机制的基本原理，为后续理解线性注意力奠定基础。

1.2 标准注意力公式

注意力机制的核心思想是：给定一组查询 (Query) 和一组键值对 (Key-Value)，计算查询与每个键的相似度，并以此作为权重对值进行加权求和。

对于序列长度为 N 、隐藏维度为 d 的输入，标准注意力的计算过程如下：

$$\mathbf{Q} = \mathbf{X}\mathbf{W}_Q, \quad \mathbf{K} = \mathbf{X}\mathbf{W}_K, \quad \mathbf{V} = \mathbf{X}\mathbf{W}_V \quad (1.1)$$

其中 $\mathbf{X} \in \mathbb{R}^{N \times d}$ 是输入序列， $\mathbf{W}_Q, \mathbf{W}_K, \mathbf{W}_V \in \mathbb{R}^{d \times d}$ 是可学习的投影矩阵。

注意力输出为：

$$\text{Attention}(\mathbf{Q}, \mathbf{K}, \mathbf{V}) = \text{softmax}\left(\frac{\mathbf{Q}\mathbf{K}^\top}{\sqrt{d}}\right) \mathbf{V} \quad (1.2)$$

其中 softmax 操作按行进行， $\sqrt{d}$ 是缩放因子。

1.3 多头注意力

Transformer 模型使用多头注意力 (Multi-Head Attention)，将查询、键、值分别投影到多个子空间中独立计算注意力，然后将结果拼接：

$$\text{MultiHead}(\mathbf{Q}, \mathbf{K}, \mathbf{V}) = \text{Concat}(\text{head}_1, \dots, \text{head}_h) \mathbf{W}_O \quad (1.3)$$

其中：

$$\text{head}_i = \text{Attention}(\mathbf{Q}\mathbf{W}_Q^i, \mathbf{K}\mathbf{W}_K^i, \mathbf{V}\mathbf{W}_V^i) \quad (1.4)$$

h 是头数， $\mathbf{W}_O$ 是输出投影矩阵。

1.4 计算复杂度分析

标准注意力的计算复杂度主要集中在 $\mathbf{QK}^\top$ 这一矩阵乘法上。对于序列长度 N 和隐藏维度 d :

- 计算 $\mathbf{QK}^\top$: $\mathcal{O}(N^2d)$
- 计算 softmax: $\mathcal{O}(N^2)$
- 计算与 $\mathbf{V}$ 的乘积: $\mathcal{O}(N^2d)$
- 总体时间复杂度: $\mathcal{O}(N^2d)$
- 总体空间复杂度: $\mathcal{O}(N^2)$ (需要存储 $N \times N$ 的注意力矩阵)

当序列长度 N 增大时,二次复杂度成为显著瓶颈。例如,当 $N = 10^5$ 时,注意力矩阵需要存储 10^{10} 个元素,这在实际中是不可行的。

1.5 长序列挑战

在实际应用中,许多场景需要处理超长序列:

- 文档级文本理解: 数万 token
- 高分辨率图像处理: 1024×1024 图像展平后超过 10^6 token
- 基因组序列分析: 百万级碱基对
- 音频信号处理: 长时间音频序列

标准注意力机制在这些场景下计算和内存开销过大,这推动了高效注意力机制的研究,其中线性注意力是最重要的方向之一。

第二章 线性注意力的动机

2.1 二次复杂度的根本问题

标准 Transformer 注意力机制的计算和存储复杂度均为序列长度的二次方。具体而言，注意力分数矩阵 $\mathbf{A} = \text{softmax}(\mathbf{QK}^\top / \sqrt{d}) \in \mathbb{R}^{N \times N}$ 的计算和存储是最主要的瓶颈。

我们可以从矩阵乘法的角度来理解这一瓶颈。设 $\mathbf{Q} \in \mathbb{R}^{N \times d}$ ， $\mathbf{K} \in \mathbb{R}^{N \times d}$ ，则 $\mathbf{QK}^\top$ 的结果是 $N \times N$ 矩阵。即使 $d \ll N$ ，当 N 很大时， N^2 的规模使得计算变得不可行。

2.2 为什么不能简单优化矩阵乘法？

一个自然的想法是： $\mathbf{QK}^\top$ 本质上就是两个矩阵相乘，是否存在更高效的计算方式？答案是：对于一般的稠密矩阵乘法， $\mathcal{O}(N^2d)$ 是最优的，无法进一步降低复杂度。

但注意，我们的最终目标并不是得到注意力分数矩阵本身，而是计算注意力输出 $\mathbf{AV}$ 。如果能够避免显式构造 $N \times N$ 的注意力矩阵，就有可能降低复杂度。

2.3 结合律的启发

考虑矩阵乘法的结合律：

$$\text{Attention}(\mathbf{Q}, \mathbf{K}, \mathbf{V}) = \underbrace{\text{softmax}\left(\frac{\mathbf{QK}^\top}{\sqrt{d}}\right)}_{\mathbf{A} \in \mathbb{R}^{N \times N}} \mathbf{V} \quad (2.1)$$

如果去掉 softmax 的非线性归一化约束，考虑一个简化的注意力形式：

$$\mathbf{AV} = (\mathbf{QK}^\top) \mathbf{V} \quad (2.2)$$

利用矩阵乘法的结合律，我们可以改变计算顺序：

$$(\mathbf{QK}^\top) \mathbf{V} = \mathbf{Q}(\mathbf{K}^\top \mathbf{V}) \quad (2.3)$$

其中：

- $\mathbf{K}^\top \mathbf{V} \in \mathbb{R}^{d \times d}$ ，计算复杂度 $\mathcal{O}(Nd^2)$

- $\mathbf{Q}(\mathbf{K}^\top \mathbf{V}) \in \mathbb{R}^{N \times d}$, 计算复杂度 $\mathcal{O}(Nd^2)$
- 总体复杂度: $\mathcal{O}(Nd^2)$

由于 $d \ll N$, $\mathcal{O}(Nd^2)$ 远优于 $\mathcal{O}(N^2d)$ 。这就是线性注意力的核心思想——通过改变计算顺序, 将复杂度从关于 N 的二次方降低到线性。

2.4 Softmax 的挑战

然而, 上述推导忽略了 softmax 函数。softmax 引入了行归一化和指数运算, 使得它无法简单地通过结合律来优化。具体来说:

$$\text{softmax} \left(\frac{\mathbf{Q}\mathbf{K}^\top}{\sqrt{d}} \right)_{ij} = \frac{\exp(\mathbf{q}_i^\top \mathbf{k}_j / \sqrt{d})}{\sum_{k=1}^N \exp(\mathbf{q}_i^\top \mathbf{k}_k / \sqrt{d})} \quad (2.4)$$

分母中的求和依赖于所有键向量 $\mathbf{k}_k$, 这导致每一行的归一化因子不同, 无法将计算分解为 $\mathbf{Q}(\mathbf{K}^\top \mathbf{V})$ 的形式。

因此, 线性注意力的关键问题变为: 如何替代 softmax, 使得注意力计算能够利用结合律改变计算顺序?

2.5 线性注意力的基本思路

线性注意力通过以下方式解决 softmax 的挑战:

1. 使用核函数 (kernel function) 替代点积相似度 $\mathbf{q}_i^\top \mathbf{k}_j$
2. 设计可分解的核函数 $\kappa(\mathbf{q}_i, \mathbf{k}_j) = \phi(\mathbf{q}_i)^\top \phi(\mathbf{k}_j)$
3. 通过特征映射 ϕ 将计算改写为:

$$\mathbf{O}_i = \frac{\sum_{j=1}^N \kappa(\mathbf{q}_i, \mathbf{k}_j) \mathbf{v}_j}{\sum_{j=1}^N \kappa(\mathbf{q}_i, \mathbf{k}_j)} = \frac{\phi(\mathbf{q}_i)^\top \sum_{j=1}^N \phi(\mathbf{k}_j) \mathbf{v}_j}{\phi(\mathbf{q}_i)^\top \sum_{j=1}^N \phi(\mathbf{k}_j)} \quad (2.5)$$

4. 令 $\mathbf{S} = \sum_{j=1}^N \phi(\mathbf{k}_j) \mathbf{v}_j^\top \in \mathbb{R}^{d \times d}$ 和 $\mathbf{z} = \sum_{j=1}^N \phi(\mathbf{k}_j) \in \mathbb{R}^d$, 这些聚合量只需一次遍历即可计算

通过这种方式, 线性注意力实现了 $\mathcal{O}(Nd^2)$ 的时间复杂度和 $\mathcal{O}(Nd)$ 的空间复杂度。

第三章 线性注意力的核心原理

3.1 核函数视角的注意力

线性注意力的核心在于将标准注意力中的 softmax 核函数替换为可分解的核函数。从核方法的视角来看，标准注意力使用的核函数为：

$$\kappa_{\text{softmax}}(\mathbf{q}, \mathbf{k}) = \frac{\exp(\mathbf{q}^\top \mathbf{k} / \sqrt{d})}{\sum_{k'} \exp(\mathbf{q}^\top \mathbf{k}' / \sqrt{d})} \quad (3.1)$$

这是一个归一化的指数核函数，分母依赖于所有键的集合，导致无法分解。

线性注意力使用一个可分解的核函数来替代：

$$\kappa(\mathbf{q}_i, \mathbf{k}_j) = \phi(\mathbf{q}_i)^\top \phi(\mathbf{k}_j) \quad (3.2)$$

其中 $\phi: \mathbb{R}^d \rightarrow \mathbb{R}^m$ 是一个特征映射函数。

3.2 线性注意力公式

使用可分解核函数后，注意力输出可以写为：

$$\mathbf{o}_i = \frac{\sum_{j=1}^N \kappa(\mathbf{q}_i, \mathbf{k}_j) \mathbf{v}_j}{\sum_{j=1}^N \kappa(\mathbf{q}_i, \mathbf{k}_j)} = \frac{\sum_{j=1}^N \phi(\mathbf{q}_i)^\top \phi(\mathbf{k}_j) \mathbf{v}_j}{\sum_{j=1}^N \phi(\mathbf{q}_i)^\top \phi(\mathbf{k}_j)} \quad (3.3)$$

利用矩阵运算的结合律，分子和分母可以分别改写为：

$$\text{分子} = \phi(\mathbf{q}_i)^\top \sum_{j=1}^N \phi(\mathbf{k}_j) \mathbf{v}_j^\top = \phi(\mathbf{q}_i)^\top \mathbf{S} \quad (3.4)$$

$$\text{分母} = \phi(\mathbf{q}_i)^\top \sum_{j=1}^N \phi(\mathbf{k}_j) = \phi(\mathbf{q}_i)^\top \mathbf{z} \quad (3.5)$$

其中：

$$\mathbf{S} = \sum_{j=1}^N \phi(\mathbf{k}_j) \mathbf{v}_j^\top \in \mathbb{R}^{m \times d}, \quad \mathbf{z} = \sum_{j=1}^N \phi(\mathbf{k}_j) \in \mathbb{R}^m \quad (3.6)$$

3.3 计算顺序的变换

标准注意力的计算流程：

$$\mathbf{O} = \underbrace{\text{softmax}(\mathbf{Q}\mathbf{K}^\top / \sqrt{d})}_{\mathbf{A} \in \mathbb{R}^{N \times N}} \mathbf{V} \quad (3.7)$$

需要先计算 $N \times N$ 的注意力矩阵，复杂度 $\mathcal{O}(N^2 d)$ 。

线性注意力的计算流程：

1. 计算 $\mathbf{S} = \sum_{j=1}^N \phi(\mathbf{k}_j) \mathbf{v}_j^\top$ ，复杂度 $\mathcal{O}(Nd^2)$
2. 计算 $\mathbf{z} = \sum_{j=1}^N \phi(\mathbf{k}_j)$ ，复杂度 $\mathcal{O}(Nd)$
3. 对每个 i ，计算 $\mathbf{o}_i = \frac{\phi(\mathbf{q}_i)^\top \mathbf{S}}{\phi(\mathbf{q}_i)^\top \mathbf{z}}$ ，总复杂度 $\mathcal{O}(Nd^2)$

总体时间复杂度为 $\mathcal{O}(Nd^2)$ ，空间复杂度为 $\mathcal{O}(Nd)$ （不需要存储 $N \times N$ 矩阵）。

3.4 特征映射的设计

特征映射 ϕ 的设计是线性注意力的关键。它需要满足以下条件：

1. 非负性： $\phi(\mathbf{q}_i)^\top \phi(\mathbf{k}_j) \geq 0$ ，以确保注意力权重非负
2. 近似指数核： $\phi(\mathbf{q}_i)^\top \phi(\mathbf{k}_j) \approx \exp(\mathbf{q}_i^\top \mathbf{k}_j / \sqrt{d})$
3. 计算高效： ϕ 的计算和存储开销应当尽可能小

不同的线性注意力方法在特征映射 ϕ 的设计上做出了不同选择。

3.4.1 elu+1 映射

Katharopoulos 等人在 Linear Transformer 中提出了简单的特征映射：

$$\phi(\mathbf{x}) = \text{elu}(\mathbf{x}) + 1 \quad (3.8)$$

其中 elu 函数定义为：

$$\text{elu}(x) = \begin{cases} x, & x > 0 \\ \alpha(\exp(x) - 1), & x \leq 0 \end{cases} \quad (3.9)$$

这个映射保证输出始终为正，且计算成本低廉。

3.4.2 随机傅里叶特征

基于 Bochner 定理，对于平移不变的核函数（如 RBF 核），可以通过随机傅里叶特征进行近似：

$$\phi(\mathbf{x}) = \frac{1}{\sqrt{m}} \begin{bmatrix} \cos(\boldsymbol{\omega}_1^\top \mathbf{x} + b_1) \\ \vdots \\ \cos(\boldsymbol{\omega}_m^\top \mathbf{x} + b_m) \end{bmatrix} \quad (3.10)$$

其中 $\boldsymbol{\omega}_i$ 从高斯分布中随机采样， b_i 从均匀分布 $[0, 2\pi]$ 中采样。

3.5 因果注意力与掩码

在自回归模型（如语言模型）中，需要确保每个位置只能关注之前的位置。在线性注意力的框架下，这可以通过累积和（cumulative sum）自然地实现。

对于因果（causal）线性注意力，每个位置 i 的聚合量为：

$$\mathbf{S}_i = \sum_{j=1}^i \phi(\mathbf{k}_j) \mathbf{v}_j^\top, \quad \mathbf{z}_i = \sum_{j=1}^i \phi(\mathbf{k}_j) \quad (3.11)$$

这可以通过递推高效计算：

$$\mathbf{S}_i = \mathbf{S}_{i-1} + \phi(\mathbf{k}_i) \mathbf{v}_i^\top, \quad \mathbf{z}_i = \mathbf{z}_{i-1} + \phi(\mathbf{k}_i) \quad (3.12)$$

这种递推形式恰好对应于序列从左到右的处理方式，无需显式构造掩码矩阵。

第四章 Linear Transformer

4.1 概述

Linear Transformer 由 Katharopoulos 等人于 2020 年提出，是线性注意力机制的开创性工作之一。该论文首次系统地将注意力计算重写为可分解的核函数形式，实现了关于序列长度的线性复杂度。

4.2 从 softmax 到可分解核

标准 Transformer 的多头自注意力定义为：

$$\text{Attention}(\mathbf{Q}, \mathbf{K}, \mathbf{V}) = \text{softmax}\left(\frac{\mathbf{Q}\mathbf{K}^\top}{\sqrt{d_k}}\right) \mathbf{V} \quad (4.1)$$

Linear Transformer 的关键洞察是：softmax 操作可以被视为核函数的归一化形式。具体地，softmax 注意力中的相似度函数为：

$$\text{sim}(\mathbf{q}, \mathbf{k}) = \exp\left(\frac{\mathbf{q}^\top \mathbf{k}}{\sqrt{d_k}}\right) \quad (4.2)$$

但这个核函数无法直接分解为 $\phi(\mathbf{q})^\top \phi(\mathbf{k})$ 的形式。Linear Transformer 使用了一个简单的替代方案：

$$\phi(\mathbf{x}) = \text{elu}(\mathbf{x}) + 1 \quad (4.3)$$

这个特征映射有两个重要性质：

1. $\phi(\mathbf{x}) > 0$ 对所有 $\mathbf{x}$ 成立，保证注意力权重非负
2. 近似捕捉输入之间的正相关性

4.3 归一化

为了替代 softmax 的自动归一化功能，Linear Transformer 引入了显式的归一化项。每个查询位置的注意力输出为：

$$\mathbf{o}_i = \frac{\sum_{j=1}^N \phi(\mathbf{q}_i)^\top \phi(\mathbf{k}_j) \mathbf{v}_j}{\sum_{j=1}^N \phi(\mathbf{q}_i)^\top \phi(\mathbf{k}_j)} \quad (4.4)$$

利用结合律重写为：

$$\mathbf{o}_i = \frac{\phi(\mathbf{q}_i)^\top \left(\sum_{j=1}^N \phi(\mathbf{k}_j) \mathbf{v}_j^\top \right)}{\phi(\mathbf{q}_i)^\top \left(\sum_{j=1}^N \phi(\mathbf{k}_j) \right)} = \frac{\phi(\mathbf{q}_i)^\top \mathbf{S}}{\phi(\mathbf{q}_i)^\top \mathbf{z}} \quad (4.5)$$

4.4 因果掩码的递推形式

对于自回归任务，Linear Transformer 利用其递推特性自然实现因果掩码。对于序列位置 i ：

$$\mathbf{o}_i = \frac{\phi(\mathbf{q}_i)^\top \mathbf{S}_i}{\phi(\mathbf{q}_i)^\top \mathbf{z}_i} \quad (4.6)$$

其中：

$$\mathbf{S}_i = \mathbf{S}_{i-1} + \phi(\mathbf{k}_i) \mathbf{v}_i^\top, \quad \mathbf{z}_i = \mathbf{z}_{i-1} + \phi(\mathbf{k}_i) \quad (4.7)$$

初始状态 $\mathbf{S}_0 = \mathbf{0}$, $\mathbf{z}_0 = \mathbf{0}$ 。

这种递推形式在训练时（非自回归）和推理时（自回归）都有效：

- 训练阶段：可以并行计算所有位置的输出，使用累积和操作
- 推理阶段：只需维护状态 $(\mathbf{S}_i, \mathbf{z}_i)$ ，每步 $\mathcal{O}(d^2)$ 计算

4.5 多头线性注意力

Linear Transformer 也支持多头机制。对于每个头 h ：

$$\text{head}_h = \text{LinearAttention}(\mathbf{Q}_h, \mathbf{K}_h, \mathbf{V}_h) \quad (4.8)$$

其中 $\mathbf{Q}_h, \mathbf{K}_h, \mathbf{V}_h$ 是通过不同的投影矩阵获得的。最终输出为所有头的拼接经过线性投影。

4.6 复杂度对比

	时间复杂度	空间复杂度
标准注意力	$\mathcal{O}(N^2 d)$	$\mathcal{O}(N^2 + Nd)$
Linear Transformer	$\mathcal{O}(Nd^2)$	$\mathcal{O}(Nd)$

表 4.6.1: 标准注意力与 Linear Transformer 复杂度对比

当 $N > d$ 时, Linear Transformer 在时间和空间上都有显著优势。

4.7 伪代码

Algorithm 1 Linear Transformer 前向传播

Require: $\mathbf{Q}, \mathbf{K}, \mathbf{V} \in \mathbb{R}^{N \times d}$, causal: bool

Ensure: $\mathbf{O} \in \mathbb{R}^{N \times d}$

```

1:  $\mathbf{Q} \leftarrow \phi(\mathbf{Q}), \mathbf{K} \leftarrow \phi(\mathbf{K})$ 
2: if causal then
3:    $\mathbf{S} \leftarrow \mathbf{0}_{d \times d}, \mathbf{z} \leftarrow \mathbf{0}_d$ 
4:   for  $i = 1$  to  $N$  do
5:      $\mathbf{S} \leftarrow \mathbf{S} + \mathbf{K}[i]^\top \mathbf{V}[i]$ 
6:      $\mathbf{z} \leftarrow \mathbf{z} + \mathbf{K}[i]$ 
7:      $\mathbf{O}[i] \leftarrow (\mathbf{Q}[i]^\top \mathbf{S}) / (\mathbf{Q}[i]^\top \mathbf{z} + \epsilon)$ 
8:   end for
9: else
10:   $\mathbf{S} \leftarrow \mathbf{K}^\top \mathbf{V}$ 
11:   $\mathbf{z} \leftarrow \sum_{i=1}^N \mathbf{K}[i]$ 
12:   $\mathbf{O} \leftarrow \mathbf{Q}\mathbf{S} / (\mathbf{Q}\mathbf{z} + \epsilon)$ 
13: end if
14: return  $\mathbf{O}$ 

```

第五章 Performer 与随机特征注意力

5.1 概述

Performer 由 Choromanski 等人于 2020 年提出，是另一种重要的线性注意力方法。与 Linear Transformer 使用 $\text{elu}+1$ 映射不同，Performer 提出了 FAVOR+ 机制 (Fast Attention Via positive Orthogonal Random features)，通过随机特征近似 softmax 核函数。

5.2 FAVOR 机制

FAVOR 的核心思想是使用随机傅里叶特征 (Random Fourier Features, RFF) 来近似 softmax 核函数。

5.2.1 softmax 核的随机特征近似

考虑 softmax 注意力中的核函数：

$$\kappa(\mathbf{q}, \mathbf{k}) = \exp(\mathbf{q}^\top \mathbf{k}) \quad (5.1)$$

这里省略了缩放因子和归一化。对于查询和键向量都经过适当归一化后，有：

$$\|\mathbf{q} - \mathbf{k}\|^2 = \|\mathbf{q}\|^2 + \|\mathbf{k}\|^2 - 2\mathbf{q}^\top \mathbf{k} \quad (5.2)$$

因此：

$$\exp(\mathbf{q}^\top \mathbf{k}) \propto \exp\left(-\frac{1}{2}\|\mathbf{q} - \mathbf{k}\|^2\right) \quad (5.3)$$

这正是高斯核 (RBF 核) 的形式。根据 Bochner 定理，高斯核可以通过随机傅里叶特征近似：

$$\exp(\mathbf{q}^\top \mathbf{k}) \approx \mathbb{E}_{\boldsymbol{\omega}} [\exp(\boldsymbol{\omega}^\top \mathbf{q}) \exp(\boldsymbol{\omega}^\top \mathbf{k})] \quad (5.4)$$

其中 $\boldsymbol{\omega} \sim \mathcal{N}(\mathbf{0}, \mathbf{I})$ 。

5.2.2 正交随机特征

FAVOR+ 的关键改进是使用正交随机特征来降低近似方差。具体地：

$$\phi(\mathbf{x}) = \frac{1}{\sqrt{m}} \begin{bmatrix} \exp(\boldsymbol{\omega}_1^\top \mathbf{x}) \\ \vdots \\ \exp(\boldsymbol{\omega}_m^\top \mathbf{x}) \end{bmatrix} \quad (5.5)$$

其中 $\boldsymbol{\omega}_i$ 通过正交化过程采样，使得它们相互正交。正交特征相比独立随机采样具有更低的方差，从而可以用更少的特征维度 m 达到相同的近似精度。

5.3 FAVOR+ 的具体实现

FAVOR+ 使用以下特征映射：

$$\phi_i(\mathbf{x}) = \frac{1}{\sqrt{m}} \exp\left(-\frac{\|\mathbf{x}\|^2}{2}\right) \exp(\boldsymbol{\omega}_i^\top \mathbf{x}), \quad i = 1, \dots, m \quad (5.6)$$

其中 $\boldsymbol{\omega}_i$ 通过以下方式生成：

1. 从标准正态分布采样 m 个向量 $\mathbf{r}_1, \dots, \mathbf{r}_m \sim \mathcal{N}(\mathbf{0}, \mathbf{I})$
2. 对它们进行 Gram-Schmidt 正交化，得到 $\boldsymbol{\omega}_1, \dots, \boldsymbol{\omega}_m$

5.4 理论保证

Performer 论文证明了以下理论结果：

- FAVOR+ 的特征映射以任意精度近似 softmax 核，所需特征数 $m = \mathcal{O}(\log N)$
- 正交随机特征的方差比独立随机特征低，近似误差的方差上界为 $\mathcal{O}(1/m)$
- 对于有界输入，FAVOR+ 注意力与标准注意力的输出差异可以被严格控制

5.5 与其他线性注意力的比较

方法	特征映射	特征数	归一化
Linear Transformer	$\text{elu}(\mathbf{x}) + 1$	d	显式
Performer (FAVOR+)	随机指数特征	$m > d$	隐式/显式

表 5.5.1: 线性注意力方法比较

Performer 的 FAVOR+ 相比 Linear Transformer 的 elu+1 映射：

- 优点：理论上更接近标准 softmax 注意力的行为
- 缺点：特征维度 m 通常大于 d ，计算开销稍大

5.6 训练技巧

Performer 在训练中使用了以下技巧来提高稳定性：

1. 重参数化：在特征映射前对输入进行归一化
2. 广义注意力框架：允许特征映射输出负值，通过额外的归一化处理
3. 混合训练：先用标准注意力预训练，再替换为线性注意力微调

第六章 其他线性注意力变体

6.1 概述

在 Linear Transformer 和 Performer 之后，研究者提出了多种线性注意力的变体和改进方法，旨在提高表达能力、数值稳定性和训练效果。本章介绍几种重要的变体。

6.2 MGA (Memory-based Global Attention)

MGA 结合了线性注意力和记忆机制，通过引入一个小型的”记忆矩阵”来增强线性注意力的表达能力。

$$\mathbf{o}_i = \text{LinearAttn}(\mathbf{q}_i, \{\mathbf{k}_j, \mathbf{v}_j\}) + \mathbf{M}^\top \mathbf{q}_i \quad (6.1)$$

其中 $\mathbf{M} \in \mathbb{R}^{d \times r}$ 是可学习的记忆矩阵， $r \ll N$ 是记忆槽的数量。这种方法允许模型在保持线性复杂度的同时，学习全局的模式表示。

6.3 GLA (Gated Linear Attention)

GLA 将门控机制引入线性注意力，以增强对信息的控制能力。其核心思想是：

$$\mathbf{S}_t = \alpha_t \odot \mathbf{S}_{t-1} + \beta_t \odot (\phi(\mathbf{k}_t) \mathbf{v}_t^\top) \quad (6.2)$$

其中 $\alpha_t, \beta_t \in \mathbb{R}^{d \times d}$ 是门控信号，由查询和输入动态生成。 $\odot$ 表示逐元素乘法。

门控机制允许模型学习何时更新状态、何时保留历史信息，类似于 LSTM 中的遗忘门和输入门。

6.4 RWKV

RWKV (Receptance Weighted Key Value) 是一种将 RNN 效率和 Transformer 质量相结合的架构。它本质上是一种特殊的线性注意力，其递推形式为：

$$\mathbf{wkv}_t = \text{diag}(\mathbf{u}) \cdot \mathbf{v}_t + \sum_{i=1}^{t-1} \text{diag}(\mathbf{w})^{t-1-i} \cdot \mathbf{v}_i \quad (6.3)$$

其中 $\mathbf{u}$ 和 $\mathbf{w}$ 是可学习的时间衰减参数。RWKV 的关键创新在于：

- 使用通道混合 (channel mixing) 替代前馈网络
- 通过时间衰减 $\mathbf{w} \in (0, 1)$ 实现类似 RNN 的遗忘机制
- 训练时通过并行扫描 (parallel scan) 实现并行化
- 推理时是纯 RNN 形式, $\mathcal{O}(1)$ 每步

6.5 Retentive Network (RetNet)

RetNet 由 Microsoft 于 2023 年提出, 核心思想是在训练时使用并行形式, 在推理时使用递归形式。

RetNet 使用 multi-scale decay 来替代标准注意力中的 softmax:

$$\mathbf{A}_{i,j} = \gamma^{i-j} \cdot \exp(\mathbf{q}_i^\top \mathbf{k}_j / \sqrt{d}), \quad j \leq i \quad (6.4)$$

其中 $\gamma \in (0, 1)$ 是衰减因子。这种设计使得:

- 训练时: 可以使用累积和并行计算所有位置的输出
- 推理时: 状态递推更新, 每步 $\mathcal{O}(d^2)$

6.6 HGRN (Hierarchically Gated RNN)

HGRN 结合了线性注意力和层次门控机制。其递推公式为:

$$\mathbf{S}_t = \mathbf{g}_t \odot \mathbf{S}_{t-1} + \tilde{\mathbf{k}}_t \tilde{\mathbf{v}}_t^\top \quad (6.5)$$

其中 $\mathbf{g}_t = \sigma(\mathbf{W}_g \mathbf{x}_t)$ 是门控信号。HGRN 通过多层堆叠实现了从局部到全局的信息整合。

6.7 基于卷积的线性注意力

一些方法将线性注意力与卷积操作结合。基本思想是:

$$\mathbf{o}_i = \sum_{j=1}^N \kappa(i-j) \cdot \phi(\mathbf{q}_i)^\top \phi(\mathbf{k}_j) \mathbf{v}_j \quad (6.6)$$

其中 $\kappa(i-j)$ 是基于位置距离的衰减函数。这种方法引入了位置感知, 弥补了纯线性注意力对位置信息敏感不足的问题。

6.8 变体比较

方法	复杂度	递推	门控	并行训练
Linear Transformer	$\mathcal{O}(Nd^2)$	✓	✗	✓
Performer	$\mathcal{O}(Nmd)$	✓	✗	✓
GLA	$\mathcal{O}(Nd^2)$	✓	✓	✓
RWKV	$\mathcal{O}(Nd)$	✓	✓	✓
RetNet	$\mathcal{O}(Nd^2)$	✓	✗	✓

表 6.8.1: 线性注意力变体特性比较

第七章 线性注意力的挑战与改进

7.1 表达能力的差距

线性注意力面临的最主要挑战是表达能力不如标准 softmax 注意力。

7.1.1 问题分析

标准 softmax 注意力能够学习尖锐的注意力分布，即对少数关键位置赋予高权重，对其他位置赋予低权重。这是因为 softmax 的指数放大效应：

$$\text{softmax}(s_i) = \frac{\exp(s_i)}{\sum_j \exp(s_j)} \quad (7.1)$$

当某个 s_i 显著大于其他值时，softmax 会赋予其接近 1 的权重。这种”聚焦”能力对许多任务至关重要。

相比之下，线性注意力使用的核函数（如 elu+1、随机特征）无法产生同样尖锐的分布。具体而言：

- elu+1 映射产生的核值范围有限，无法像指数函数那样放大差异
- 随机特征近似引入了近似误差，尤其在极端值情况下

7.1.2 实验观察

研究者发现线性注意力在以下场景下与标准注意力差距较大：

- 需要精确检索的任务（如 copying task）
- 需要关注长距离依赖的任务
- 需要尖锐注意力分布的任务

7.2 数值稳定性问题

线性注意力在训练中可能遇到数值稳定性问题。

7.2.1 分母接近零

在归一化公式中：

$$\mathbf{o}_i = \frac{\phi(\mathbf{q}_i)^\top \mathbf{S}}{\phi(\mathbf{q}_i)^\top \mathbf{z}} \quad (7.2)$$

如果分母 $\phi(\mathbf{q}_i)^\top \mathbf{z}$ 接近零，会导致数值不稳定。实践中通常添加一个小常数 ϵ ：

$$\mathbf{o}_i = \frac{\phi(\mathbf{q}_i)^\top \mathbf{S}}{\phi(\mathbf{q}_i)^\top \mathbf{z} + \epsilon} \quad (7.3)$$

但这只是部分缓解，不能完全解决问题。

7.2.2 状态矩阵的累积

在递推形式中，状态矩阵 $\mathbf{S}_t = \sum_{j=1}^t \phi(\mathbf{k}_j) \mathbf{v}_j^\top$ 会随着序列长度增长而累积，可能导致数值溢出或精度损失。

7.3 改进方法

7.3.1 混合注意力

一种实用的策略是混合使用标准注意力和线性注意力：

$$\mathbf{O} = \lambda \cdot \text{SoftmaxAttn}(\mathbf{Q}, \mathbf{K}, \mathbf{V}) + (1 - \lambda) \cdot \text{LinearAttn}(\mathbf{Q}, \mathbf{K}, \mathbf{V}) \quad (7.4)$$

或者将序列分为两部分：局部窗口内使用标准注意力，全局使用线性注意力。

7.3.2 改进的特征映射

CosFormer

CosFormer 使用余弦函数来重构特征映射：

$$\phi(\mathbf{x}) = \text{relu}(\mathbf{x}) \odot \cos\left(\frac{\pi \cdot \text{pos}}{M}\right) \quad (7.5)$$

其中位置编码通过余函数引入，使得近位置的核值更大，远位置自然衰减。

NormFormer

NormFormer 通过输出归一化来稳定训练：

$$\mathbf{O} = \text{LayerNorm}(\text{LinearAttn}(\mathbf{Q}, \mathbf{K}, \mathbf{V})) \quad (7.6)$$

7.3.3 训练策略改进

课程学习

一种有效的训练策略是：

1. 先用标准 softmax 注意力预训练模型
2. 然后替换为线性注意力，用较小的学习率微调
3. 逐渐增加序列长度

这种方法可以让模型先在标准注意力下学习良好的表示，再适应线性注意力的近似。

蒸馏

使用标准注意力模型作为教师模型，线性注意力模型作为学生模型进行知识蒸馏：

$$\mathcal{L} = \alpha \mathcal{L}_{\text{task}} + (1 - \alpha) \text{KL}(\mathbf{O}_{\text{teacher}} \| \mathbf{O}_{\text{student}}) \quad (7.7)$$

7.4 理论分析

7.4.1 近似误差界

设标准注意力输出为 $\mathbf{O}^*$ ，线性注意力输出为 $\hat{\mathbf{O}}$ ，则近似误差可以界定为：

$$\|\mathbf{O}^* - \hat{\mathbf{O}}\|_F \leq C \cdot \max_i \|\phi(\mathbf{q}_i) - \phi^*(\mathbf{q}_i)\| \cdot \|\mathbf{V}\|_F \quad (7.8)$$

其中 ϕ^* 是理想的特征映射， C 是常数。

7.4.2 表达能力分析

有研究表明，线性注意力的表达能力受限于特征映射的秩。具体地，线性注意力可以表示的函数空间维度为 $\mathcal{O}(md)$ ，其中 m 是特征维度， d 是隐藏维度。

相比之下，标准注意力可以表示的函数空间维度为 $\mathcal{O}(Nd)$ ，当 $N \gg m$ 时，标准注意力具有更强的表达能力。

7.5 实践建议

根据现有的研究和实践经验，以下建议可以帮助更好地使用线性注意力：

1. 对于需要长序列处理的场景，线性注意力是可行的选择
2. 使用混合注意力或局部窗口策略来弥补表达能力的不足

3. 在训练时添加梯度裁剪和层归一化来提高稳定性
4. 特征映射的选择应根据任务特性调整: elu+1 简单高效, 随机特征更接近 softmax
5. 推理时利用递推形式实现 $\mathcal{O}(1)$ 每 token 的高效生成

第八章 总结与展望

8.1 线性注意力的核心贡献

线性注意力机制通过将标准 softmax 注意力重写为可分解的核函数形式，成功地将注意力计算的时间复杂度从 $\mathcal{O}(N^2d)$ 降低到 $\mathcal{O}(Nd^2)$ ，空间复杂度从 $\mathcal{O}(N^2)$ 降低到 $\mathcal{O}(Nd)$ 。这使得 Transformer 类模型能够处理更长的序列。

线性注意力的核心思想可以概括为以下三个步骤：

1. 将 softmax 核函数替换为可分解的核函数 $\kappa(\mathbf{q}, \mathbf{k}) = \phi(\mathbf{q})^\top \phi(\mathbf{k})$
2. 利用矩阵乘法的结合律，将计算顺序从 $(\mathbf{Q}\mathbf{K}^\top)\mathbf{V}$ 改为 $\mathbf{Q}(\mathbf{K}^\top\mathbf{V})$
3. 通过累积聚合量 $\mathbf{S} = \sum \phi(\mathbf{k}_j)\mathbf{v}_j^\top$ 和 $\mathbf{z} = \sum \phi(\mathbf{k}_j)$ 实现线性复杂度

8.2 主要方法回顾

方法	特征映射	时间复杂度	关键特性
Linear Transformer	$\text{elu}(\mathbf{x}) + 1$	$\mathcal{O}(Nd^2)$	开创性工作，简单高效
Performer	随机傅里叶特征	$\mathcal{O}(Nmd)$	理论保证，接近 softmax
GLA	$\text{elu}+1$ + 门控	$\mathcal{O}(Nd^2)$	门控增强信息控制
RWKV	时间衰减	$\mathcal{O}(Nd)$	RNN 推理效率
RetNet	多尺度衰减	$\mathcal{O}(Nd^2)$	训练推理双模式

表 8.2.1: 主要线性注意力方法总结

8.3 与 Mamba/SSM 的关系

近年来，Mamba 等状态空间模型（SSM）也在长序列建模领域取得了显著进展。线性注意力与 SSM 有密切的联系：

- 两者都具有 $\mathcal{O}(N)$ 或 $\mathcal{O}(Nd)$ 的递推形式

- 线性注意力的状态矩阵 $\mathbf{S}_t$ 类似于 SSM 的隐藏状态
- 最新的 Mamba-2 工作表明，SSM 可以被视为一种特殊的线性注意力
- 两者的界限正在逐渐模糊，出现了统一的视角

8.4 未来方向

线性注意力的研究仍有以下值得探索的方向：

1. **更好的特征映射**：设计更接近 softmax 且计算高效的特征映射，减少表达能力差距
2. **理论分析**：深入理解线性注意力的表达能力边界和最优特征映射
3. **混合架构**：探索标准注意力与线性注意力的最优组合方式
4. **硬件优化**：针对线性注意力的计算模式设计专用的硬件加速方案
5. **多模态应用**：将线性注意力扩展到视觉、音频、多模态等领域
6. **统一框架**：建立线性注意力、SSM 和其他高效注意力方法的统一理论框架

8.5 结语

线性注意力是大模型高效架构设计的重要方向之一。随着理论认识的深入和新方法的出现，线性注意力有望在保持高效的同时，进一步缩小与标准注意力的性能差距，成为处理超长序列的标配方案。